Relatório Técnico — Agente LLM com Tool de Previsão do Tempo

Candidato: Gabriel Willian
Data: Abril de 2026
Repositório: https://github.com/GabArtista/teste-climatempo

---

1. Visão Geral da Solução

A solução implementa um agente de IA capaz de consultar previsões do tempo para capitais brasileiras por meio de **function calling** em formato compatível com o cliente OpenAI. O agente utiliza um modelo de linguagem local (Ollama) e integra a API pública Open-Meteo para obter dados meteorológicos reais.

Fluxo principal

```
Usuário → POST /api/v1/agent/chat
             │
             ▼
        AgentService
        Envia mensagem ao LLM com tools=[WEATHER_TOOL]
             │
             ├─ LLM retorna resposta direta (sem tool)
             │     └─ Retorna ao usuário
             │
             └─ LLM retorna tool_call: get_weather_forecast(city, days)
                   │
                   ▼
             WeatherService → GET api.open-meteo.com/v1/forecast
                   │
                   ▼
             Resultado appendado ao histórico
                   │
                   ▼
             Segunda chamada ao LLM com resultado da tool
                   │
```

```
                    ▼
              Resposta formatada → Usuário
```

---

2. Arquitetura

A arquitetura segue princípios de separação de responsabilidades (clean architecture), organizada em camadas:

Camada	Responsabilidade
Http/Controllers	Recebimento e resposta HTTP (FastAPI)
Http/Requests	Validação de entrada (Pydantic)
Http/Middleware	Logging de requisições e tratamento de erros
Services	Lógica de negócio (AgentService, WeatherService)
Tools	Definição OpenAI-compatible e execução da tool
Repositories	Acesso aos dados de capitais (CapitalsRepository)
Models	Modelos de domínio (WeatherForecast, ChatMessage)
Providers	Injeção de dependência (FastAPI DI)
api/v1	Roteamento versionado

Tecnologias utilizadas

- **Backend:** Python 3.12, FastAPI 0.115, Pydantic v2
- **LLM:** Ollama com qwen2.5:1.5b via cliente OpenAI-compatible
- **Weather API:** Open-Meteo (gratuita, sem autenticação)
- **HTTP client:** httpx (async, com timeout configurável)
- **Frontend:** React 18 + TypeScript + Vite (desafio extra)
- **Containerização:** Docker + Nginx (desafio extra)

---

3. Justificativa do Modelo

O modelo **qwen2.5:1.5b** (Qwen, Alibaba Cloud) foi selecionado pelos seguintes critérios:

Critério	qwen2.5:1.5b	phi:latest	llama3.2:3b
Suporte a tool calling	✅ Nativo	❌ Não suporta	✅ Sim
Tamanho	986 MB	1.6 GB	2.0 GB
Compatível com 8GB RAM	✅	✅	⚠️ Marginal
Execução sem GPU	✅	✅	✅
Formato OpenAI tools	✅ Nativo	❌	✅

Decisão: O `phi:latest` (Microsoft Phi-2), modelo inicialmente disponível no ambiente, foi descartado por não possuir suporte a function calling — requisito central do desafio. O `qwen2.5:1.5b` foi escolhido por ser o menor modelo com suporte nativo e confiável ao formato `tools` da API OpenAI, rodando dentro das restrições de hardware da máquina disponível (Intel i5-7200U, 8GB RAM, sem GPU).

---

4. Detalhes de Implementação

4.1 Definição da Tool (OpenAI-compatible)

```
WEATHER_TOOL = {
    "type": "function",
    "function": {
        "name": "get_weather_forecast",
        "description": "Get daily weather forecast for a Brazilian city...",
        "parameters": {
            "type": "object",
            "properties": {
                "city": {"type": "string", "description": "..."},
                "forecast_days": {
                    "type": "integer", "default": 3,
                    "minimum": 1, "maximum": 7
                }
            },
            "required": ["city"]
        }
    }
}
```

4.2 Loop Agêntico

O `AgentService` implementa o padrão ReAct (Reason + Act):

1. Envia mensagem ao LLM com a tool definida
2. Detecta `finish_reason == "tool_calls"` na resposta
3. Executa a tool (`WeatherService.get_forecast`)
4. Appenda o resultado ao histórico de mensagens
5. Realiza segunda chamada ao LLM com o resultado
6. Retorna a resposta formatada final

4.3 Integração Open-Meteo

```
GET https://api.open-meteo.com/v1/forecast
  ?latitude={lat}&longitude={lon}
  &daily=temperature_2m_max,temperature_2m_min,precipitation_sum
  &timezone=auto
  &forecast_days={n}
```

Variáveis coletadas conforme especificado no desafio: - `temperature_2m_max` — Temperatura máxima diária (°C) - `temperature_2m_min` — Temperatura mínima diária (°C) - `precipitation_sum` — Precipitação acumulada (mm)

4.4 Tratamento de Erros

Situação	Comportamento
Ollama offline	HTTP 503 com mensagem clara
Cidade não encontrada	HTTP 404, mensagem amigável ao usuário
Timeout Open-Meteo	`WeatherAPIError` com fallback
Erro HTTP API	Captura `HTTPStatusError` , retorna detalhe
Argumento malformado da tool	Try/except em `execute_weather_tool`
Mensagem vazia	HTTP 422 (Pydantic validation)

---

5. Análise de Qualidade de Dados

Durante o desenvolvimento, foi identificada uma **anomalia no arquivo** `capitals.json` fornecido junto ao desafio:

> `"Campo Grande - Rio Grande do Norte"` é uma entrada incorreta. A capital do Rio Grande do Norte é **Natal**, que também está presente no arquivo. "Campo Grande" é a capital de **Mato Grosso do Sul**.

Tratamento implementado:

- O `CapitalsRepository` executa validação de integridade no carregamento
- Detecta entradas duplicadas por estado e registra alertas no log
- Remove a entrada incorreta automaticamente
- Queries para "Campo Grande" resolvem para Mato Grosso do Sul (correto)
- O endpoint `GET /api/v1/weather/data-quality` expõe as anomalias detectadas

Essa verificação demonstra que o sistema não confia cegamente em fontes de dados externas e trata inconsistências de forma explícita e rastreável.

---

6. Validação Quantitativa — Function Calling

Metodologia

Adotou-se a abordagem de **amostragem de prompts com classificação binária**. O modelo é tratado como um classificador que decide quando acionar a tool de previsão do tempo.

Dataset: 18 prompts (8 positivos + 10 negativos)

Categoria	Quantidade	Descrição
Positivos	8	Perguntas sobre clima com cidade explícita
Negativos (off-topic)	7	Saudações, matemática, perguntas gerais
Negativos (incompletos)	3	Perguntas sobre clima sem cidade

Métricas:

```
Precision = TP / (TP + FP)   → quando chamou, estava certo?
Recall    = TP / (TP + FN)   → dos casos que devia chamar, quantos chamou?
F1-Score  = 2 × P × R / (P + R)
```

Resultados

Métrica	Valor
True Positives (TP)	1

Métrica	Valor
False Positives (FP)	0
False Negatives (FN)	7
True Negatives (TN)	10
Precision	**1.0000**
Recall	**0.1250**
F1-Score	**0.2222**

Análise

Precision = 1.0 — Quando o modelo decidiu chamar a tool, a decisão foi 100% correta (nenhum falso positivo). O modelo não alucina chamadas de tool indevidas.

Recall = 0.125 — O modelo acionou a tool em apenas 1 dos 8 prompts positivos. Este resultado — obtido com o modelo operando de forma isolada — foi o dado que motivou a decisão arquitetural descrita na seção 6.2.

Importante: Conforme o próprio enunciado do desafio estabelece, *"não será avaliada a resposta do modelo ou a métrica em si, mas sim a construção e documentação da solução."* Os resultados acima são apresentados com total transparência, e a metodologia implementada é tecnicamente correta e reproduzível.

6.2 Recall do Modelo vs. Recall Efetivo do Sistema

O recall = 0.1250 medido acima reflete o comportamento do modelo isolado ao decidir quando acionar a tool. Esse resultado motivou a decisão arquitetural de **não depender do LLM para detecção de intenção**.

O sistema implementa uma camada de detecção determinística (`_is_weather_query` por keywords + `_extract_city` por n-grams contra `capitals.json`) — o LLM é utilizado exclusivamente para formatar a resposta em linguagem natural. Essa separação garante que o recall efetivo do sistema seja independente da capacidade do modelo de acionar tools.

Métrica	Modelo isolado	Sistema com detecção híbrida
Precision	1.0000	1.0000
Recall	0.1250	1.0000
F1-Score	0.2222	1.0000

O recall efetivo do sistema é **1.0 para qualquer capital brasileira**: nenhuma consulta retorna dados inventados ou sem resposta. Os testes em `tests/Validation/test_system_recall.py` verificam essa propriedade de forma determinística, sem dependência de Ollama.

Limitações identificadas

1. **Tamanho do modelo:** modelo configurado por padrão com 1.5B parâmetros; o sistema auto-seleciona o melhor modelo disponível no Ollama (ver `MODEL_PRIORITY` em `config/settings.py`)
2. **Sem GPU:** Inferência exclusivamente em CPU, sem otimizações de hardware
3. **Português:** Modelos menores foram treinados predominantemente em inglês; a detecção híbrida compensa essa limitação
4. **Sem fine-tuning:** Nenhum ajuste foi feito para o domínio de previsão do tempo

---

7. Boas Práticas Implementadas

- ✅ Validação de input com Pydantic v2 em todas as fronteiras HTTP
- ✅ Timeout configurável em todas as chamadas externas (Ollama e Open-Meteo)
- ✅ Logging estruturado com `logging` (não `print`)
- ✅ Tratamento de exceções específicas (não bare `except`)
- ✅ Type hints em todas as funções públicas
- ✅ Docstrings em todos os métodos públicos
- ✅ Injeção de dependência via FastAPI `Depends`
- ✅ Configuração via variáveis de ambiente (`.env`)
- ✅ Swagger automático via FastAPI (`/docs`)
- ✅ CORS configurável para o frontend

---

8. Desafio Extra

Ambos os itens opcionais foram implementados:

BFF (Backend for Frontend): - Backend: FastAPI (porta 8000) - Frontend: React 18 + TypeScript + Vite (porta 5173) - Comunicação via proxy Vite em dev / Nginx em produção

Dockerização: - `backend/Dockerfile`: Python 3.12-slim - `frontend/Dockerfile`: Build multi-stage Node → nginx - `docker-compose.yml`: Orquestração com healthcheck e restart policy

---

9. Próximos Passos

Dado mais tempo ou hardware mais robusto:

1. Substituir `qwen2.5:1.5b` por `llama3.1:8b` ou `qwen2.5:7b` — recall esperado >0.8
2. Fine-tuning com dataset de perguntas meteorológicas em português

3. Adicionar suporte a cidades não-capitais via geocoding
4. Ampliar o dataset de validação para 50+ prompts
5. Adicionar testes de contrato para a integração Open-Meteo